

Starvation and Livelock

Unit 4 – Deadlock Modeling and System-level Handling

Prashant Jadav

DEPSTAR – CHARUSAT

Learning Objectives

- Define starvation and livelock in concurrent operating systems.
- Identify common causes of starvation.
- Explain how aging improves scheduling fairness.
- Distinguish starvation, deadlock, and livelock.
- Analyze livelock using process/resource interaction scenarios.
- Identify techniques for reducing starvation and livelock.

Starvation: Definition

Starvation

Starvation (indefinite blocking) occurs when a process waits for an unbounded amount of time because other processes are repeatedly given preference.

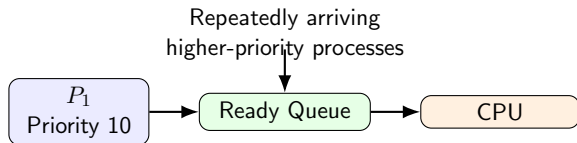
- The system continues to make progress.
- The affected process does not receive the CPU or required resource.
- It is commonly associated with unfair scheduling or resource allocation.

Simple example

A low-priority process is ready to execute, but a continuous stream of high-priority processes keeps receiving the CPU first.

How Starvation Occurs in Priority Scheduling

- Assume lower numerical value means higher priority.
- P_1 has priority 10.
- New processes with priorities 1–5 continuously arrive.
- The scheduler always selects the highest-priority ready process.
- P_1 may remain in the Ready queue indefinitely.



Key point

Starvation is possible when the scheduling policy does not guarantee bounded waiting or sufficient fairness.

Scheduling Policies That May Cause Starvation

- **Priority scheduling:** low-priority processes may be repeatedly bypassed.
- **SJF:** a long job may wait if shorter jobs continuously arrive.
- **SRTF/SRTF-like policies:** a process may repeatedly be preempted by jobs with shorter remaining times.
- **Multilevel Queue:** a lower-priority queue may receive little or no CPU service if higher-priority queues remain busy.

Design question

A scheduling algorithm may optimize average waiting time while still providing poor fairness to an individual process.

Starvation and Bounded Waiting

Bounded Waiting

There should be a finite bound on the number of times other processes can enter a critical section after a process has requested entry and before that process is admitted.

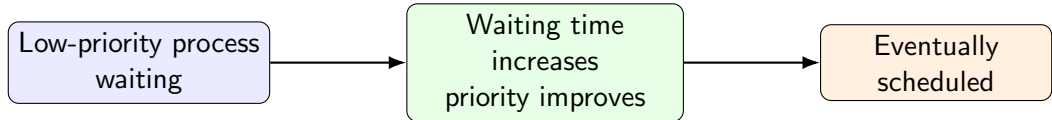
- Bounded waiting is an important fairness property.
- If no meaningful bound exists, indefinite waiting can occur.
- FIFO queues and fair lock acquisition can reduce starvation.

Fairness $\implies$ Reduced risk of starvation

Aging: Solution to Starvation

Aging

Aging gradually improves the priority of a process that has been waiting for a long time.



- Aging converts waiting time into increasing scheduling priority.
- It improves long-term fairness.

Example of Aging

Suppose priority values range from 0 (highest) to 100 (lowest).

Waiting time	Priority	Interpretation
0 min	80	Low priority
10 min	70	Priority improved
20 min	60	Priority improved
30 min	50	Increasing chance of service

Effect

As a process waits longer, its priority moves toward the high-priority range, reducing the possibility of indefinite postponement.

Other Ways to Reduce Starvation

- Use **fair scheduling** policies.
- Maintain **FIFO ordering** for resource or lock requests where appropriate.
- Use **aging** in priority-based scheduling.
- Provide **bounded waiting** guarantees for synchronization.
- Avoid continuously favoring the same class of processes.

Design trade-off

Improving fairness may increase the waiting time of some high-priority or short jobs. OS design must balance fairness with responsiveness and throughput.

Livelock: Definition

Livelock

Livelock occurs when processes remain active and repeatedly change their state or actions in response to one another, but the system makes no useful progress.

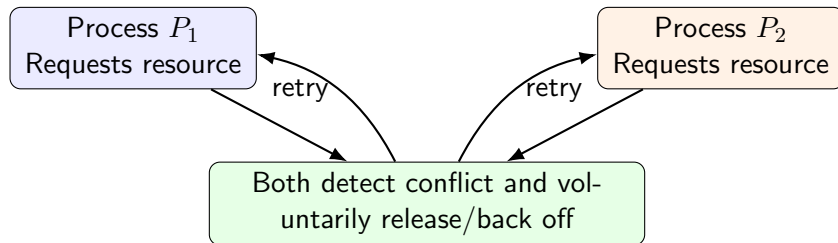
- Processes are not permanently blocked.
- Processes continue executing.
- Their actions repeatedly interfere with one another.
- Useful work does not complete.

Key idea

Deadlock: stuck and waiting

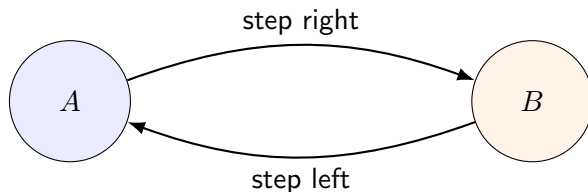
Livelock: active but not progressing

Livelock Example: Two Processes



- Both processes repeatedly respond to the other.
- Each retry may trigger another conflict.
- CPU activity continues, but useful work is not completed.

Real-world Analogy of Livelock



Two people meet in a narrow hallway.
Both repeatedly move aside
in response to the other.
They remain active, but neither passes.

OS interpretation

The same pattern can occur when concurrent processes repeatedly release, retry, yield, or

Causes of Livelock

- ① **Excessive resource preemption:** processes repeatedly release and request resources.
- ② **Symmetric back-off:** processes make identical decisions at the same time.
- ③ **Repeated retry:** failed operations are immediately retried without sufficient variation.
- ④ **Aggressive recovery:** rollback/retry mechanisms repeatedly undo useful progress.
- ⑤ **Poor coordination:** processes respond to one another without a mechanism for breaking symmetry.

Livelock vs. Deadlock

Feature	Deadlock	Livelock
Activity	Processes are blocked	Processes remain active
Progress	No progress	No useful progress
State	Waiting states persist	States/actions repeatedly change
CPU activity	May be low	May remain high
Typical cause	Circular waiting	Repeated mutual responses
Example	P_1 waits for P_2 and vice versa	P_1 and P_2 repeatedly yield/retry

Starvation vs. Livelock

Feature	Starvation	Livelock
Main problem	One process is repeatedly denied service	Processes repeatedly act without progress
System activity	Other processes may progress normally	Involved processes remain active
Typical cause	Unfair scheduling/resource allocation	Repeated responses/retries
Typical example	Low-priority process never scheduled	Two processes repeatedly back off
Mitigation	Aging, fairness, bounded waiting	Coordination, randomized back-off, controlled retries

Starvation, Deadlock and Livelock

Deadlock

Processes wait indefinitely for one another

Starvation

One process waits indefinitely while others progress

Livelock

Processes remain active but make no progress

Diagnostic question

Ask two questions:

- 1 Is the process/system **blocked or active**?
- 2 Is the system making **useful progress**?

- 1 A low-priority process remains in the Ready queue while higher-priority jobs continuously arrive.

Classification: Starvation

- 2 P_1 holds R_1 and waits for R_2 , while P_2 holds R_2 and waits for R_1 .

Classification: Deadlock

- 3 Two processes repeatedly detect a conflict, release their resources, and immediately retry.

Classification: Livelock

How to Control Livelock

- Introduce **randomized back-off** so competing processes do not retry simultaneously.
- Limit the number or frequency of retries.
- Use explicit coordination or ownership rules.
- Avoid symmetric decisions that cause repeated conflict.
- Detect repeated unsuccessful cycles and change strategy.

Principle

A livelock solution must break the repeated interaction pattern and allow one or more processes to make actual progress.

Exam-oriented Questions

- 1 Explain starvation and give one CPU scheduling example.
- 2 Why can SJF or priority scheduling cause starvation?
- 3 Explain aging and show how it reduces starvation.
- 4 Differentiate deadlock and livelock using process activity and system progress.
- 5 Two processes repeatedly release and reacquire a resource without completing their work. Identify the condition and justify your answer.
- 6 Suggest two mechanisms for reducing livelock.

Key Takeaways

- **Starvation** = indefinite postponement of a process because it is repeatedly bypassed.
- **Aging** = gradually improving the priority of a waiting process to improve fairness.
- **Livelock** = processes remain active but repeatedly fail to make useful progress.
- Starvation is primarily associated with **unfair service**.
- Livelock is primarily associated with **repeated interaction without progress**.
- Good OS design considers both **progress** and **fairness**.

Remember

Deadlock: waiting without progress
Starvation: one process waits while others progress
Livelock: active behavior without useful progress